A Subexponential Algorithm for #k-SAT

Eitan Porat

Abstract

This note is a short sketch of the algorithm in the Anthropic manuscript [3]. For every $\delta > 0$, the algorithm counts the satisfying assignments of a k-CNF formula on n variables in time $O_{k,\delta}(2^{\delta n})$. In particular it solves k-SAT in this time, and so it refutes the Exponential Time Hypothesis. The note is a rewrite of that argument, prepared with the aid of AI. It simplifies the proof, and it aims to make it compact and understandable to a general audience.

The algorithm divides the clauses into groups, which have intra-group edges and inter-group edges. The cost is set by the inter-group edges: the *shared variables*. So the goal is to remove shared variables. We arrange the groups in a square grid. Merging the groups of each row, or of each column, removes the variables shared inside a row, but most shared variables join groups in different rows and different columns, and no merge removes them. The key tool is a simple algebraic identity that rewrites two such crossing connections as connections inside columns minus connections inside rows. With it, every level of the recursion removes a constant fraction of the shared variables.

Contents

1 The Problem **2**

2 Sparsification **3**

3 #k-SAT as a Sum of Products **4**

4 Groups of Clauses **4**

5 The Cost of a Grouping **5**

6 Shared Variables Cannot Be Avoided **5**

7 Balanced Groups **5**

8 Merging Rows and Columns **6**

9 Turning Diagonals into Row and Column Edges **8**

10 Full Crossings **10**

11 Merging Many Rectangles at Once **13**

12 Recursion **14**

A Proof of the Sparsification Lemma **15**

A.1 The Algorithm 15

A.2 Analysis 17

A.3 Thresholds and the Number of Leaves 18

A.4 Proof of the Lemma 19

Running times.

We write $f = O(g)$ if $f \leq c\,g$ for a constant c, $f = \Omega(g)$ if $f \geq c\,g$ for a constant $c > 0$, and $f = \Theta(g)$ if both hold. A subscript names what the constant may depend on: in $O_{k,\delta}$ it may depend on k and δ, but not on n.

1 The Problem

Formulas.

Let $x_1, \ldots, x_n$ be Boolean variables. A **literal** is a variable x_i or its negation $\neg x_i$. A **clause** is an OR of literals, for example $x_1 \vee \neg x_2 \vee x_3$. A **$k$-CNF formula** is an AND of clauses $C_1, \ldots, C_m$, each with at most k literals, for example

$$F = (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_4).$$

An **assignment** $a \in \{0,1\}^n$ gives x_i the value a_i (1 is true and 0 is false). It **satisfies** F if every clause of F has a true literal. Let $Z(F)$ be the number of satisfying assignments of F.

Problems.

- k**-SAT**: given a k-CNF formula F, decide whether $Z(F) > 0$.
- #k**-SAT**: given a k-CNF formula F, compute $Z(F)$.

An algorithm for #k-SAT also solves k-SAT.

The question.

Brute force tries all 2^n assignments. It was conjectured that 3-SAT cannot be solved much faster, that is, not in time below exponential:

Exponential Time Hypothesis (ETH) (Impagliazzo and Paturi [5])**.** *There is a constant $c > 0$ such that no algorithm solves 3-SAT on n variables in time 2^{cn}.*

Its counting version, #ETH, makes the same claim for #3-SAT (Dell et al. [4]). We show the opposite: for every $\delta > 0$ there is an algorithm for #k-SAT, and so for k-SAT, with running time $O_{k,\delta}(2^{\delta n})$, that is, at most $c(k,\delta) \cdot 2^{\delta n}$. So ETH and #ETH are false.

2 Sparsification

We would like to assume that the formula does not have many clauses: $m = O(n)$. The sparsification lemma lets us do this.

The sparsification lemma.

Lemma 1 (Sparsification lemma)**.** *For every $k \geq 2$ and every $\varepsilon > 0$ there is a number $M = M(k,\varepsilon)$ such that every k-CNF formula F on n variables can be split, in time $2^{\varepsilon n}\mathrm{poly}_{k,\varepsilon}(n)$, into a list $\mathcal{L}$ of k-CNF formulas on the same variables such that*

(a) $\mathcal{L}$ has at most $2^{\varepsilon n}$ formulas;

(b) every formula in $\mathcal{L}$ has at most Mn clauses;

(c) every assignment that satisfies F satisfies exactly one formula in $\mathcal{L}$, and every assignment that satisfies a formula in $\mathcal{L}$ satisfies F.

In particular, $Z(F) = \sum_{F' \in \mathcal{L}} Z(F')$.

Impagliazzo, Paturi and Zane [6] proved the lemma with "at least one formula" in (c), which is enough for k-SAT, and Calabro, Impagliazzo and Paturi [2] gave a sharper version. Dell et al. [4] proved the form above, in which the formulas in $\mathcal{L}$ have disjoint sets of solutions; this is the form that counting needs. Appendix A gives a self-contained proof.

The reduction.

Fix $\delta > 0$ and apply the lemma with $\varepsilon = \delta/2$; this fixes $M = M(k, \delta/2)$. Suppose that every k-CNF formula with n variables and at most Mn clauses can be counted in time $O_{k,\delta}(2^{\delta n/2})$. Then

$$Z(F) = \sum_{F' \in \mathcal{L}} Z(F'), \qquad |\mathcal{L}| \le 2^{\delta n/2},$$

takes time $2^{\delta n/2}\,\mathrm{poly}(n) + |\mathcal{L}| \cdot O_{k,\delta}(2^{\delta n/2}) = O_{k,\delta}(2^{\delta n})$. So it is enough to count formulas with at most Mn clauses, and from now on $m \le Mn$, that is, $m = O_{k,\delta}(n)$.

3 #k-SAT as a Sum of Products

Let $C_i(a) = 1$ if the assignment a satisfies C_i, and $C_i(a) = 0$ otherwise. Then

$$Z(F) = \sum_{a \in \{0,1\}^n} \prod_{i=1}^{m} C_i(a).$$

If no two clauses share a variable, the sum factors: $Z(F) = \prod_i \bigl(2^{|C_i|} - 1\bigr)$, times 2 for each variable in no clause. All the difficulty comes from the shared variables.

4 Groups of Clauses

Divide the clauses into groups $g = 1, \ldots, G$. Give each variable one copy for each two consecutive clauses that contain it, with a check that the copies agree, and from now on call these copies the variables. So each variable is one bit, and it is either

- **inside** one group, or
- **shared** by exactly two groups.

Summing the inside variables first gives

$$Z(F) = \sum_{\text{shared variables}} \prod_{g=1}^{G} Z_g,$$

where Z_g counts the assignments of the variables inside group g that satisfy group g, for the given values of the shared variables. We call the groups, with their functions Z_g and the shared variables, a **network** $\mathcal{N}$; its value $Z(\mathcal{N})$ is the sum above.

5 The Cost of a Grouping

Let N be the number of shared variables. For each of their 2^N values we compute the G numbers Z_g by brute force and multiply them. A group has $O_k(n/G)$ variables inside it, and G is a constant (Section 12), so we get a running time of

$$2^{N+O_k(n/G)}.$$

6 Shared Variables Cannot Be Avoided

Can a clever grouping make the number of shared variables small? In general, no. For example, in a random 3-CNF with $m = Mn$ clauses the clauses and their shared variables form an expander, so every grouping into G groups of equal size shares a constant fraction of the variables, so $N = \Theta(n)$.

7 Balanced Groups

We cannot make the shared variables few, but we can spread them evenly. A theorem of Bollobás and Scott [1, Theorems 4–5] gives, in polynomial time, a grouping in which every group has $m/G \pm O_k(1)$ clauses and every two groups share $t \pm O_k(1)$ variables, where

$$t = \Theta_k\Big(\frac{n}{G^2}\Big).$$

8 Merging Rows and Columns

Put the G groups in a $\sqrt{G} \times \sqrt{G}$ grid; $g_{r,c}$ is the group in row r and column c, and $Z_{r,c}$ is its count $Z_{g_{r,c}}$. Seen from one group, the other $G-1$ groups are of three kinds (Figure 1).

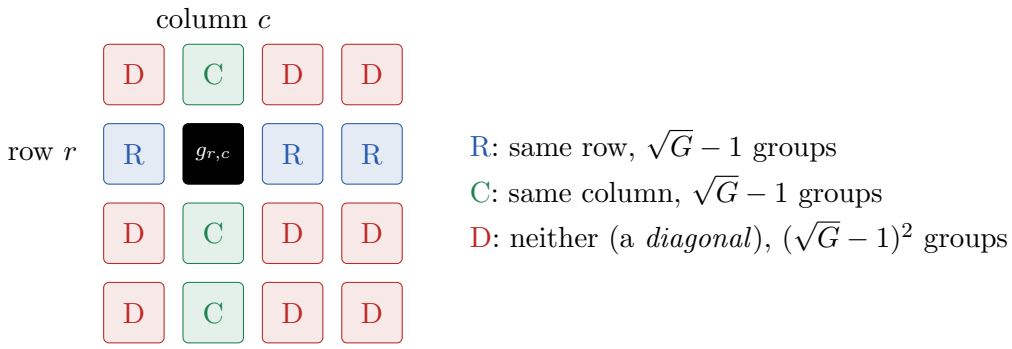


Figure 1: The other groups, seen from the group $g_{r,c}$ in a 4×4 grid.

Every two groups share $t \pm O_k(1)$ variables (Section 7), so, up to this $\pm O_k(1)$, the N shared variables divide like the pairs:

kind of pair	number of pairs	fraction of the shared variables
R: same row	$\frac{1}{2}G(\sqrt{G}-1)$	$\frac{1}{\sqrt{G}+1}$
C: same column	$\frac{1}{2}G(\sqrt{G}-1)$	$\frac{1}{\sqrt{G}+1}$
D: diagonal	$\frac{1}{2}G(\sqrt{G}-1)^2$	$\frac{\sqrt{G}-1}{\sqrt{G}+1}$

For large G, almost all shared variables are on diagonals.

Merging the rows.

Write Z with the shared variables sorted by kind and the product sorted by rows:

$$Z = \sum_{C \text{ variables}} \sum_{D \text{ variables}} \sum_{R \text{ variables}} \prod_{r=1}^{\sqrt{G}} \prod_{c=1}^{\sqrt{G}} Z_{r,c}.$$

An R variable of row r is shared by two groups of row r, so only the factors $Z_{r,1}, \dots, Z_{r,\sqrt{G}}$ use it. So we can change the order of summation and move its sum inside, next to these factors:

$$Z = \sum_{C \text{ variables}} \sum_{D \text{ variables}} \prod_{r=1}^{\sqrt{G}} \left[\sum_{R \text{ variables of row } r} \prod_{c=1}^{\sqrt{G}} Z_{r,c} \right].$$

This is equivalent to merging the groups of each row into one group (Figure 2): the bracket of row r is the count of the merged group.

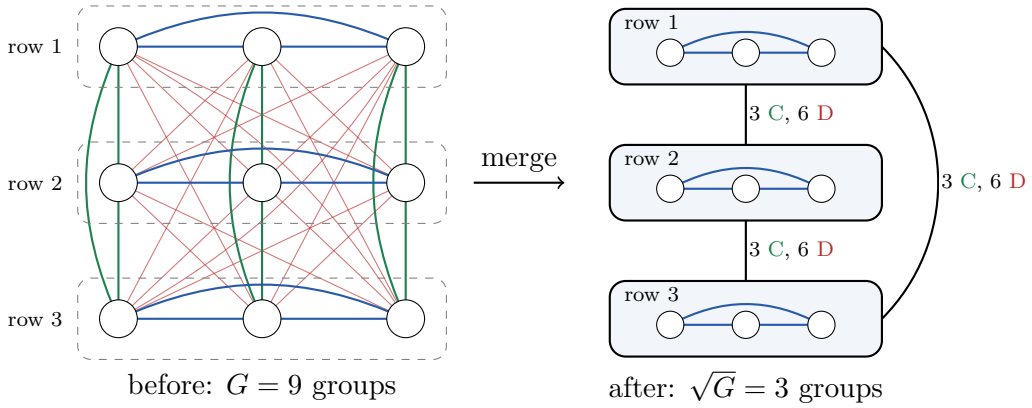


Figure 2: Merging the rows of a 3×3 grid. Left: all 36 pairs of groups, namely 9 R pairs (blue), 9 C pairs (green) and 18 D pairs (red). Right: each row is one new group (a box) that contains its three groups and its 3 R pairs. Every two new groups are joined by 3 C pairs and 6 D pairs; in general, by $\sqrt{G}$ C pairs and $\sqrt{G}(\sqrt{G}-1)$ D pairs.

The run time.

Before the merge, the outer sum runs over all N shared variables. After it, the outer sum runs over the C and D variables only. For each of their values, the $\sqrt{G}$ brackets are computed one after the other, and each bracket runs over the R variables of one row: $1/\sqrt{G}$ of the R variables. With the fractions of the table, and up to the factor $2^{O_k(n/G)}$ of the groups (Section 5), the run times are

$$\text{before the merge:} \quad 2^{\left(\frac{1}{\sqrt{G}+1}+\frac{1}{\sqrt{G}+1}+\frac{\sqrt{G}-1}{\sqrt{G}+1}\right)N} = 2^N,$$

$$\text{after the merge:} \quad 2^{\left(\frac{1}{\sqrt{G}+1}+\frac{\sqrt{G}-1}{\sqrt{G}+1}\right)N} \cdot 2^{\frac{1}{\sqrt{G}(\sqrt{G}+1)}N} = 2^{(1-O(1/\sqrt{G}))N}.$$

Example 1. Take the clauses $C_1 = x_1 \vee x_2 \vee x_3$, $C_2 = x_1 \vee x_4 \vee x_5$, $C_3 = x_2 \vee x_4 \vee x_6$ and $C_4 = x_3 \vee x_5 \vee x_6$, so that every two of them share one variable.

Put C_1, C_2 in the top row and C_3, C_4 in the bottom row (Figure 3). The R variables are x_1, x_6, the C variables are x_2, x_5, and the D variables are x_3, x_4.

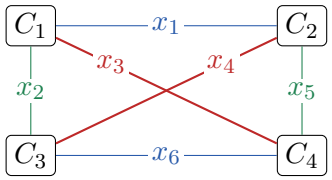


Figure 3: The example as a 2×2 grid.

Moving the sums of the R variables inside merges the rows:

$$Z = \sum_{x_1,\ldots,x_6} C_1C_2C_3C_4 = \sum_{x_2,x_3,x_4,x_5} \Big[\sum_{x_1} C_1C_2\Big]\Big[\sum_{x_6} C_3C_4\Big].$$

A merge of the columns leaves x_1, x_3, x_4, x_6 shared; a merge along the diagonals, $\{C_1, C_4\}$ and $\{C_2, C_3\}$, leaves x_1, x_2, x_5, x_6. Every merge leaves 4 shared variables.

Why merges are not enough.

We would like each step to improve the exponent by a constant fraction, independent of G. A merge improves it only by a fraction $O(1/\sqrt{G})$: a fraction $1 - O(1/\sqrt{G})$ of all shared variables are D variables, and no merge removes them. This is why we remove the diagonals next.

9 Turning Diagonals into Row and Column Edges

Take two rows $r \neq r'$ and two columns $c \neq c'$. The four groups $g_{r,c}, g_{r,c'}, g_{r',c}, g_{r',c'}$ form a **rectangle**, and its two diagonals, $g_{r,c}$–$g_{r',c'}$ and $g_{r,c'}$–$g_{r',c}$, are D pairs that cross. Every D pair is a diagonal of exactly one rectangle, so we can remove the D variables one rectangle at a time. In this section each diagonal carries one bit; Section 10 does the full crossing.

Diagonals against row and column edges.

After the merge of the rows, the D variables are in the outer sum (Section 8). Suppose, *hypothetically*, that each diagonal were an edge inside one row. Then the same merge of the rows, and no other merge, would put the D variables inside the brackets, and each row would get only its own share:

$$Z = \sum_{C \text{ variables}} \prod_{r=1}^{\sqrt{G}} \Bigg[\sum_{R \text{ variables of row } r} \; \sum_{D \text{ variables of row } r} \; \prod_{c=1}^{\sqrt{G}} Z_{r,c} \Bigg].$$

When the D variables are in the outer sum, the running time is

$$2^{(\frac{1}{\sqrt{G}+1} + \frac{\sqrt{G}-1}{\sqrt{G}+1})N} \cdot 2^{\frac{1}{\sqrt{G}(\sqrt{G}+1)}N} = 2^{(1-O(1/\sqrt{G}))N}.$$

When they are row edges, each row gets $1/\sqrt{G}$ of them, and we get

$$2^{\frac{1}{\sqrt{G}+1}N} \cdot 2^{(\frac{1}{\sqrt{G}(\sqrt{G}+1)} + \frac{\sqrt{G}-1}{\sqrt{G}(\sqrt{G}+1)})N} = 2^{O(N/\sqrt{G})}.$$

This improves the exponent from almost N to $O(N/\sqrt{G})$, and the case of a merge of the columns is the same. This motivates us to turn the diagonal edges into row and column edges.

Copies at the ends of the diagonals.

We want to separate the two ends of each diagonal. So we replace the variable of a diagonal by two new variables, one at each end, and add a new factor that forces them to be equal, the **equality condition**:

- first diagonal: x at $g_{r,c}$ and v at $g_{r',c'}$, with the condition $[x = v]$;
- second diagonal: y at $g_{r,c'}$ and u at $g_{r',c}$, with the condition $[y = u]$.

Together they give the factor

$$q = [x = v] \cdot [y = u].$$

Each condition is an edge between two groups in different rows and different columns (Figure 4), so no merge removes it.

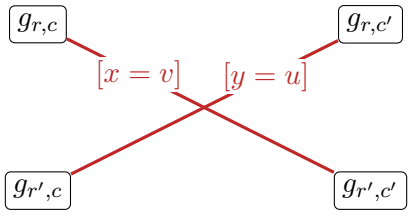


Figure 4: The two diagonals of a rectangle, as the edges $[x = v]$ and $[y = u]$.

Two diagonals are two column edges minus two row edges.

The product of the two diagonals is a difference of two products of edges (Figure 5):

$$\underbrace{[x = v] \cdot [y = u]}_{\text{two diagonal edges}} = \underbrace{[x = u] \cdot [y = v]}_{L\text{: two column edges}} - \underbrace{(y - x) \cdot (v - u)}_{T\text{: two row edges}}.$$

The factor $[x = u]$ joins $g_{r,c}$ and $g_{r',c}$, and $[y = v]$ joins $g_{r,c'}$ and $g_{r',c'}$: two edges inside the columns c and c'. The factor $(y - x)$ joins $g_{r,c}$ and $g_{r,c'}$, and $(v - u)$ joins $g_{r',c}$ and $g_{r',c'}$: two edges inside the rows r and r'. We call this identity the **crossing identity**.

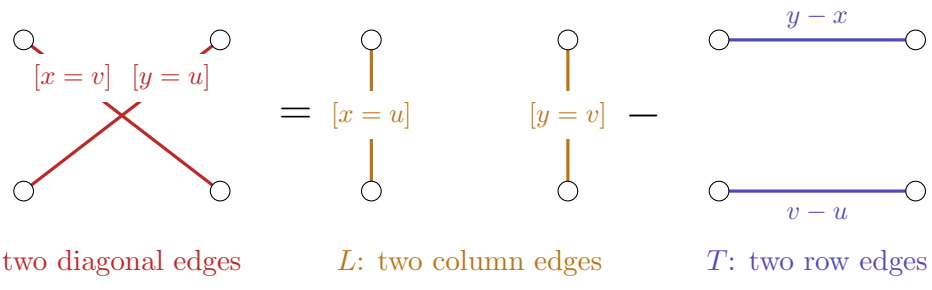


Figure 5: Two diagonal edges equal two column edges minus two row edges. The corners are $g_{r,c}, g_{r,c'}$ (top) and $g_{r',c}, g_{r',c'}$ (bottom).

Put into the sum, the crossing identity splits the value of the network in two: $Z(\mathcal{N}) = Z(\mathcal{N}_L) - Z(\mathcal{N}_T)$, where $\mathcal{N}_L$ has the two column edges and is merged by columns, and $\mathcal{N}_T$ has the two row edges and is merged by rows.

10 Full Crossings

Every two groups share $t = \Theta_k(n/G^2)$ variables (Section 7), so each diagonal of a rectangle carries t bits, not one. The copies are $x, y, u, v \in \{0,1\}^t$, and the crossing identity of Section 9 applies to each bit j:

$$q = [x = v]\,[y = u] = \prod_{j=1}^{t} [x_j = v_j]\,[y_j = u_j] = \prod_{j=1}^{t} (L_j - T_j).$$

Only one direction.

A merge puts every group into exactly one new group: $g_{r,c}$ goes with its row or with its column, not with both: both together would put all G groups into one group, with an inside cost of $2^{\Theta(n)}$. So a network is merged either by rows or by columns, never partly by each.

Row rank and column rank.

The **row rank** of a function $\varphi(x, y, u, v)$ is the rank of its matrix with rows (x, y) and columns (u, v): the smallest number s such that φ is a sum of s **top·bottom products** $f(x, y)\,h(u, v)$. Its **column rank** is the rank of its matrix with rows (x, u) and columns (y, v): the smallest s such that φ is a sum of s **left·right products** $f(x, u)\,h(y, v)$. With the rows merged, each top·bottom product splits between the brackets of rows r and r', so a factor costs its row rank; with the columns merged, it costs its column rank.

Ranks multiply over the bits. If φ_1 and φ_2 use different bits, the matrix of $\varphi_1\varphi_2$ is the Kronecker product of the matrices of φ_1 and φ_2, and $\operatorname{rank}(A \otimes B) = \operatorname{rank} A \cdot \operatorname{rank} B$. So the row rank of $\varphi_1\varphi_2$ is the product of the row ranks, and the column rank of $\varphi_1\varphi_2$ is the product of the column ranks.

Notice that $T = (y - x) \cdot (v - u)$ is one top·bottom product, and $L = [x = u] \cdot [y = v]$ is one left·right product: T has row rank 1 and L has column rank 1. The other way around they need four products: L has row rank 4 and T has column rank 4.

Mixed terms.

Multiplying out q gives one term for each set J of the bits that take T:

$$q = \sum_{J \subseteq \{1,\dots,t\}} (-1)^{|J|}\, T_J\, L_{\bar{J}}, \qquad T_J = \prod_{j \in J} T_j, \quad L_{\bar{J}} = \prod_{j \notin J} L_j.$$

Since ranks multiply over the bits, the term of J has row rank $4^{t-|J|}$ and column rank $4^{|J|}$. A pure term is cheap in one direction: all T for the rows, all L for the columns. But most terms are mixed, and a mixed term is expensive in both directions. Even if each term goes to its cheaper direction, the terms with $|J| \approx t/2$ cost about 2^t each, and there are about 2^t of them:

$$\frac{4^t}{2t+2} \;\le\; \sum_J 4^{\min(|J|,\, t-|J|)} \;\le\; 4^t.$$

This is smaller than the $4^t = 2^{2t}$ values of the $2t$ diagonal bits only by a factor polynomial in t: no exponential improvement.

Two identities.

Goal. For the rows, L is expensive (row rank 4) and T is cheap (row rank 1). We want to replace L by something cheaper, and pay only with factors T. For the columns, the roles of L and T are exchanged.

The identities.

$$L = \mu + \tfrac{1}{2}T, \qquad T = \nu + \tfrac{1}{2}L, \qquad \text{where} \quad \mu = L - \tfrac{1}{2}T, \quad \nu = T - \tfrac{1}{2}L.$$

The claim. μ has row rank 3, and ν has column rank 3.

Proof. Order the rows and the columns of each matrix as $00, 01, 10, 11$.

- Rows: with rows (x, y) and columns (u, v),

$$L = I = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad T = aa^{\mathsf{T}} = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & -1 & 0 \\ 0 & -1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \qquad a = \begin{pmatrix} 0 \\ 1 \\ -1 \\ 0 \end{pmatrix}.$$

  Since $a^{\mathsf{T}}a = 2$, $\frac{1}{2}aa^{\mathsf{T}}$ is a projection of rank 1, so $\mu = I - \frac{1}{2}aa^{\mathsf{T}}$ has rank 3.

- Columns: with rows (x, u) and columns (y, v),

$$L = ee^{\mathsf{T}} = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 \end{pmatrix}, \qquad T = P = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & -1 & 0 \\ 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix}, \qquad e = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 1 \end{pmatrix}.$$

P is orthogonal and $Pe = e$, so $\nu = P - \frac{1}{2}ee^\mathsf{T} = P\,(I - \frac{1}{2}ee^\mathsf{T})$ has rank 3.

For any factor other than $\frac{1}{2}$, $I - \lambda aa^\mathsf{T}$ and $I - \lambda ee^\mathsf{T}$ are invertible, so $\frac{1}{2}$ is the only factor that lowers the rank.

Use.

- Rows: write each L as $\mu + \frac{1}{2}T$. A factor μ costs 3 instead of 4, and a factor T costs 1. Since ranks multiply, a product with a factors μ and all other factors T has row rank 3^a.
- Columns: write each T as $\nu + \frac{1}{2}L$. A factor ν costs 3 instead of 4, and a factor L costs 1. A product with a factors ν and all other factors L has column rank 3^a.

Split by majority.

Using μ for all of q does not help: $q = \prod_j(\mu_j - \frac{1}{2}T_j)$ still has row rank 4^t: multiplied out, it is $(3+1)^t$ products, and the terms with many μ are expensive. So send each term to its majority: a term with more T than L goes to the rows, and the others go to the columns,

$$q = \underbrace{\sum_{|J|>t/2} (-1)^{|J|}\, T_J\, L_{\bar{J}}}_{q^+\text{: rows}} + \underbrace{\sum_{|J|\le t/2} (-1)^{|J|}\, T_J\, L_{\bar{J}}}_{q^-\text{: columns}}.$$

Theorem 2 (Splitting a crossing)**.** *q^+ has row rank at most $(2\sqrt{3})^t$, and q^- has column rank at most $(2\sqrt{3})^t$.*

Proof. We use two facts about ranks: the rank of a sum is at most the sum of the ranks, and ranks multiply over the bits. For a set K of bits, write $\mu_{\bar{K}} = \prod_{j\notin K} \mu_j$ and $\nu_{\bar{K}} = \prod_{j\notin K} \nu_j$.

The rows: q^+. Replace every L_j by $\mu_j + \frac{1}{2}T_j$:

$$q^+ = \sum_{|J|>t/2} (-1)^{|J|}\, T_J \prod_{j\notin J} \big(\mu_j + \tfrac{1}{2}T_j\big).$$

Multiply out the product. Each bit $j \notin J$ gives either μ_j or $\frac{1}{2}T_j$, so there is one term for each set $A \subseteq \bar{J}$ of the bits that give $\frac{1}{2}T_j$:

$$T_J \prod_{j\notin J} \big(\mu_j + \tfrac{1}{2}T_j\big) = \sum_{A\subseteq \bar{J}} 2^{-|A|}\, T_{J\cup A}\, \mu_{\overline{J\cup A}}.$$

Each term has T at the bits of $K = J \cup A$ and μ at the other bits. Collect the terms with the same K. For them $|A| = |K| - |J|$ and $|K| \geq |J| > t/2$, so

$$q^+ = \sum_{|K|>t/2} c_K\, T_K\, \mu_{\bar{K}}, \qquad c_K = \sum_{J\subseteq K,\ |J|>t/2} (-1)^{|J|}\, 2^{|J|-|K|}.$$

Then

$$\text{row rank of } q^+ \ \leq \sum_{|K|>t/2} \underbrace{1}_{T_K} \cdot \underbrace{3^{t-|K|}}_{\mu_{\bar{K}}} \leq \underbrace{2^t}_{\text{sets } K} \cdot 3^{t/2} = (2\sqrt{3})^t.$$

The columns: q^-. Similarly, with L and T exchanged: replace every T_j by $\nu_j + \frac{1}{2}L_j$. This gives $q^- = \sum_{|K|\geq t/2} c'_K\, L_K\, \nu_{\bar{K}}$ for some numbers c'_K, so q^- has column rank at most $(2\sqrt{3})^t$. □

Run time.

For one rectangle, the factor in the run time from its $2t$ diagonal bits is

$$\begin{aligned} \text{diagonals in the outer sum:} &\quad 2^{2t}, \\ q^+\text{, rows merged:} &\quad (2\sqrt{3})^t = 2^{(1+\frac{1}{2}\log_2 3)\,t} \approx 2^{1.79\,t}, \\ q^-\text{, columns merged:} &\quad (2\sqrt{3})^t \approx 2^{1.79\,t}. \end{aligned}$$

11 Merging Many Rectangles at Once

There are $\binom{\sqrt{G}}{2}^2$ rectangles. Every D variable is now two copies, and the crossing q_ρ of a rectangle ρ checks that its copies agree (Section 9). So

$$Z(\mathcal{N}) = \sum_{R,\,C,\,\text{copies}} \prod_g Z_g \prod_\rho q_\rho.$$

Put $q_\rho = q_\rho^+ + q_\rho^-$ (Theorem 2) and multiply out the product over ρ. This gives one term for each choice of a sign $\sigma(\rho) \in \{+,-\}$ for every rectangle ρ:

$$Z(\mathcal{N}) = \sum_\sigma Z(\mathcal{N}^\sigma), \qquad Z(\mathcal{N}^\sigma) = \sum_{R,\,C,\,\text{copies}} \prod_g Z_g \prod_\rho q_\rho^{\sigma(\rho)}.$$

These are $2^{\binom{\sqrt{G}}{2}^2}$ networks, a constant number, and we compute each of them separately. Each $\mathcal{N}^\sigma$ votes: if $\sigma(\rho) = +$ for at least half of the rectangles, it is merged by rows; otherwise, by columns.

The run time.

Take $\mathcal{N}^\sigma$ merged by rows. Before the split, each rectangle has its $2t$ diagonal bits in the outer sum: 2^{2t} values. After it,

$$\begin{aligned} &\text{type } + : \quad (2\sqrt{3})^t = 2^{2t} \cdot \left(\tfrac{\sqrt{3}}{2}\right)^t \text{ values (the choice of a product } f\,h), \\ &\text{type } - : \quad 2^{2t} \text{ values, as before.} \end{aligned}$$

By the vote, at least half of the rectangles have type $+$, so on average each rectangle gains at least the geometric mean $\left(\frac{\sqrt{3}}{2}\right)^{t/2}$, that is $\left(\frac{\sqrt{3}}{2}\right)^{1/4}$ for each of its $2t$ diagonal bits. The diagonal bits are the D variables, a fraction $\frac{\sqrt{G}-1}{\sqrt{G}+1}$ of the N shared variables (Section 8). So the number of values in the outer sum is

$$\begin{aligned} &\text{before the split:} \quad 2^N, \\ &\text{after the split:} \quad 2^{(1-\eta)N}, \qquad \eta = \frac{\sqrt{G}-1}{\sqrt{G}+1} \cdot \frac{1}{4} \log_2 \frac{2}{\sqrt{3}} \approx \frac{1}{2} - \frac{1}{4}\log_2(2\sqrt{3}) \approx 0.05. \end{aligned}$$

12 Recursion

A merge takes G groups to $\sqrt{G}$ groups, and then we can merge again. Let Q be the number of groups at the end and ℓ the number of merges, and start with $G = Q^{2^\ell}$ groups:

$$G = Q^{2^\ell} \;\to\; Q^{2^{\ell-1}} \;\to\; \cdots \;\to\; Q^2 \;\to\; Q.$$

Every level has at least Q^2 groups, and Q is large, so $\eta \approx \frac{1}{2} - \frac{1}{4}\log_2(2\sqrt{3})$ at every level. So after ℓ merges the outer sums keep at most $(1-\eta)^\ell N + O(1)$ bits. Take ℓ large enough that

$$(1-\eta)^\ell N + O(1) \;\le\; \frac{\delta n}{4}.$$

Such an ℓ depends only on k and δ: every variable has fewer copies than clauses that contain it, so $N \le km$, and $m = O_{k,\delta}(n)$ by Section 2. Then choose Q so large that the inside cost of the Q final groups, $2^{O_k(n/Q)}$, is at most $2^{\delta n/4}$. Altogether, the running time is

$$\underbrace{2^{\delta n/2}}_{\text{formulas in } \mathcal{L}} \cdot \underbrace{O_{k,\delta}(1)}_{\text{networks } \mathcal{N}^\sigma} \cdot \underbrace{2^{\delta n/4}}_{\text{outer sums}} \cdot \underbrace{2^{\delta n/4}}_{\text{inside the } Q \text{ groups}} = O_{k,\delta}\big(2^{\delta n}\big),$$

plus $2^{\delta n/2}\,\mathrm{poly}(n)$ to compute $\mathcal{L}$ (Section 2).

A Proof of the Sparsification Lemma

A.1 The Algorithm

Key idea: for all $\kappa < s$, find a sub-clause H with κ literals that is contained in ***many*** s-clauses of F, and split into two cases. Let $S \subseteq F$ be the s-clauses that contain H.

- If H is true, every clause of S is true: replace them all by the one clause H.
- If H is false, every literal of H is false: add the 1-clauses $\{\neg\ell\}$, $\ell \in H$, and remove H from each clause of S.

This gives the two formulas

$$F_1 = (F \setminus S) \cup \{H\}, \qquad F_2 = (F \setminus S) \cup \{\, C \setminus H : C \in S \,\} \cup \{\, \{\neg\ell\} : \ell \in H \,\}.$$

Lemma 3 (Disjoint branches, as in [4, Lemma A.1])**.** *Every assignment that satisfies F satisfies exactly one of F_1 and F_2, and every assignment that satisfies F_1 or F_2 satisfies F. So $Z(F) = Z(F_1) + Z(F_2)$. Also,* Dedup *(below) keeps the satisfying assignments of every formula; so $Z(\textsc{Dedup}(F)) = Z(F)$, and the above holds after* Dedup *is applied to F, F_1 and F_2.*

Proof. An assignment that satisfies F_1 makes H true, and one that satisfies F_2 makes every literal of H false; so no assignment satisfies both. Let α satisfy F. If α satisfies H, it satisfies F_1. If not, every literal of H is false under α, so α satisfies each $\{\neg\ell\}$, and for each $C \in S$ a literal of $C \setminus H$ is true; thus α satisfies F_2. Conversely, $H \subseteq C$ and $C \setminus H \subseteq C$ for every $C \in S$, so an assignment that satisfies F_1 or F_2 satisfies every clause of S, and hence F. Finally, Dedup removes only clauses that contain another clause of the formula; such a clause is true whenever the smaller one is, so Dedup keeps the satisfying assignments. □

Example 2. Let $F = (x \vee y \vee a) \wedge (x \vee y \vee b) \wedge (x \vee y \vee c) \wedge (x \vee d \vee e)$, let $H = x \vee y$, and let S be the first three clauses. Then

$$F_1 = (x \vee y) \wedge (x \vee d \vee e), \qquad F_2 = (\neg x) \wedge (\neg y) \wedge (a) \wedge (b) \wedge (c) \wedge (x \vee d \vee e).$$

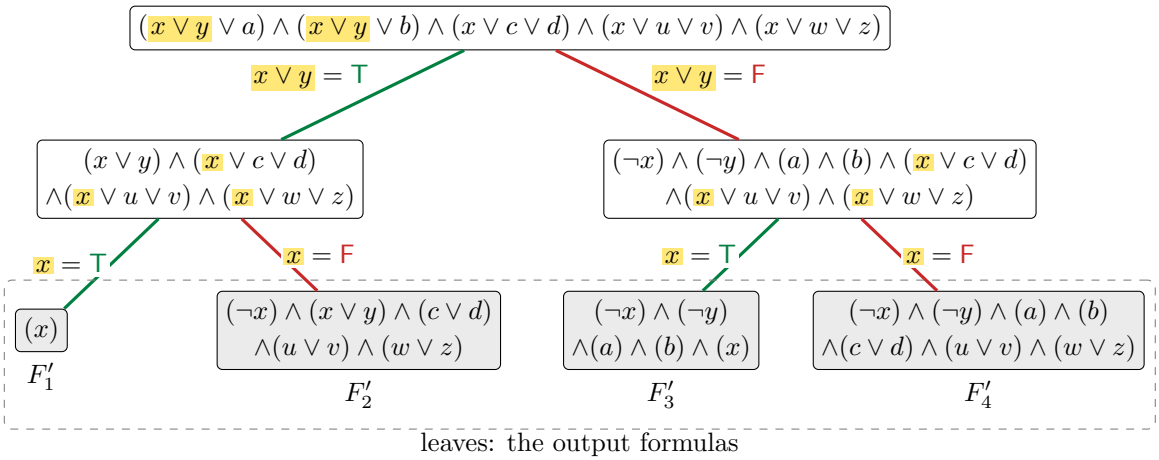
In F_1 three 3-clauses became one 2-clause; in F_2 they became three 1-clauses, and F_2 also has the 1-clauses $\neg x$ and $\neg y$.

Fix thresholds $\theta_1, \dots, \theta_{k-1}$; Section A.3 chooses them. For a formula F, let

$$\textsc{Dedup}(F) = \{\, C \in F : \text{no } D \in F \text{ with } D \subsetneq C \,\}.$$

Sparsification. $\textsc{Sparsify}(F)$, for a k-CNF formula F.

$F \leftarrow \textsc{Dedup}(F)$
for $s = 2, \dots, k$ **do** ▷ shortest clauses first
  for $\kappa = s-1, \dots, 1$ **do** ▷ largest sets first
    $H \leftarrow$ a set of κ literals in the most s-clauses of F
    $S \leftarrow \{\, C \in F : |C| = s,\ H \subseteq C \,\}$
    if $|S| \geq \theta_{s-\kappa}$ **then**
      $F_1 \leftarrow \textsc{Dedup}\big(F \cup \{H\}\big)$ ▷ $H = \mathsf{T}$
      $F_2 \leftarrow \textsc{Dedup}\big(F \cup \{\, C \setminus H : C \in S \,\} \cup \{\, \{\neg \ell\} : \ell \in H \,\}\big)$ ▷ $H = \mathsf{F}$
      return $\textsc{Sparsify}(F_1) \cup \textsc{Sparsify}(F_2)$
    end if
  end for
end for
return $\{F\}$ ▷ no set is in many clauses



$$Z(F) = Z(F'_1) + Z(F'_2) + Z(F'_3) + Z(F'_4)$$

Figure 6: The recursive calls of the sparsification algorithm ($\theta_1 = 2$, $\theta_2 = 3$).

A.2 Analysis

Let $\varepsilon > 0$. A list for a smaller ε also satisfies Lemma 1 for ε, so we may assume that ε is small enough in terms of k. We prove that there are thresholds $\theta_1(\varepsilon), \ldots, \theta_{k-1}(\varepsilon)$ such that every leaf formula has at most $M(\varepsilon)\, n$ clauses and there are at most $2^{\varepsilon n}$ leaves.

Claim 4 (Small formulas)**.** *Every leaf formula has at most Mn clauses, where*

$$M = M(k, \varepsilon) = O(1 + \theta_1 + \cdots + \theta_{k-1}).$$

Proof. At a leaf F', every literal is in fewer than θ_{s-1} s-clauses, otherwise the recursion would split F'. Count the pairs (literal ℓ, s-clause that contains ℓ): each s-clause is in s pairs, and each of the $2n$ literals is in fewer than θ_{s-1}, so

$$s \cdot \#s\text{-clauses} \;<\; 2n\,\theta_{s-1}.$$

With $\#1$-clauses $\le 2n$, this gives

$$|F'| = \sum_{s=1}^{k} \#s\text{-clauses} = O\big(1 + \theta_1 + \cdots + \theta_{k-1}\big)\, n. \qquad \square$$

Claim 5 (Few leaves)**.** *The tree has at most $2^{\varepsilon n}$ leaves.*

Fix a path from the root to a leaf. A split happens in the algorithm at some values of s and κ: H has κ literals and S is a set of s-clauses. We call it a split *on size s*. It *adds* H (case $H = \mathsf{T}$, a green edge in Figure 6), or the clauses $C \setminus H$, $C \in S$, and the 1-clauses $\{\neg\ell\}$, $\ell \in H$ (case $H = \mathsf{F}$, a red edge); these have κ, $s - \kappa$ and 1 literals, all fewer than s. A clause that is already in the formula is not added. Let A_r be the number of r-clauses added on the path.

Lemma 6 (The order)**.** *Let $1 \le j < r \le k$. After the first split on a size larger than r, every set J of j literals is always inside fewer than $2\theta_{r-j}$ r-clauses.*

Proof. Only splits on sizes $s > r$ add r-clauses, so between two such splits the number of r-clauses that contain J can only go down. So it is enough to look at one split on a size $s > r$, with a sub-clause H of κ literals, and to show two things.

1. *Before the split, J is in fewer than θ_{r-j} r-clauses.* The algorithm tested the r-clauses before the s-clauses, and found no set of j literals in θ_{r-j} or more of them.

2. *The split adds at most θ_{r-j} r-clauses that contain J.*

   (a) Case $H = \mathsf{T}$: the split adds one clause.

   (b) Case $H = \mathsf{F}$: a new r-clause $C \setminus H$ (so $s - \kappa = r$) that contains J comes from an s-clause $C \supseteq H \cup J$. The algorithm tested the larger set $H \cup J$ before H, so fewer than $\theta_{s-\kappa-j} = \theta_{r-j}$ s-clauses contain it.

Together, J is in fewer than $2\theta_{r-j}$ r-clauses after the split. □

Proposition 7 (Few short clauses)**.** *For $s = 1, \dots, k-1$ there is a number b_s, which is a function of $\theta_1, \dots, \theta_{s-1}$ only, such that $A_s \le b_s n$ on every path.*

Proof. By induction on s.

Base case. $A_1 \le 2n$, so $b_1 = 2$: there are $2n$ 1-clauses, and DEDUP never removes a 1-clause, so each of them is added at most once.

Step, $s \ge 2$. An added s-clause is in the leaf formula, or it was removed. So

$$A_s \le (s\text{-clauses in the leaf}) + (\text{added } s\text{-clauses that were removed}).$$

In the leaf there are fewer than $\theta_{s-1} n$ s-clauses (proof of Claim 4). DEDUP removes an s-clause only when a clause D strictly inside it is added, with $|D| = j < s$. The s-clause was added at a split on a size larger than s, so by Lemma 6 with $r = s$, D is inside fewer than $2\theta_{s-j}$ s-clauses when it is added. So

$$A_s \le \theta_{s-1} n + 2 \sum_{j=1}^{s-1} \theta_{s-j} A_j \le \underbrace{\left(\theta_{s-1} + 2 \sum_{j=1}^{s-1} \theta_{s-j} b_j\right)}_{b_s} n,$$

and b_s is a function of $\theta_1, \dots, \theta_{s-1}$, because each b_j with $j < s$ is. □

A.3 Thresholds and the Number of Leaves

Since b_s is a function of $\theta_1, \dots, \theta_{s-1}$, we can choose the thresholds one at a time:

$$\theta_s = \lceil b_s / \varepsilon^2 \rceil \qquad (s = 1, \dots, k-1).$$

Size of b_s. Since $b_j \ge 1 \ge \varepsilon^2$, we have $\theta_j = O(b_j/\varepsilon^2)$. For $s \ge 2$, b_s is a sum of $O(k)$ terms, and each of them is $O(b_{s-j} b_j / \varepsilon^2)$ for some $1 \le j < s$ (for the term θ_{s-1}, because $b_1 \ge 1$); so $b_s \le O(k/\varepsilon^2) \max_{1 \le j < s} b_{s-j} b_j$. By induction on s, with the same constant in O at every step, taken at least 1, $b_s \le O(k/\varepsilon^2)^{2s-1} = (k/\varepsilon)^{O(s)}$. So $b = b_1 + \cdots + b_{k-1} \le (k/\varepsilon)^{O(k)}$.

Proof of Claim 5 (Few leaves). Fix a path from the root to a leaf.

- *Splits.* Each split adds at least one clause with fewer than k literals, so the path has at most $A_1 + \cdots + A_{k-1} \le bn$ splits.
- *Red edges.* A red edge whose split adds clauses $C \setminus H$ with s literals adds at least θ_s of them (one clause $C \setminus H$ for each $C \in S$), and all the splits together add at most $A_s \le b_s n$ s-clauses. So there are at most $b_s n/\theta_s \le \varepsilon^2 n$ such red edges, and at most $(k-1)\varepsilon^2 n \le k\varepsilon^2 n$ red edges in all.

So a path is a string over $\{\mathsf{T}, \mathsf{F}\}$ (one letter for each split: T for a green edge, F for a red edge) of length at most bn with at most $k\varepsilon^2 n$ letters F. Different leaves give different strings, because their paths separate at a split where one takes T and the other F; so we may pad every string with T's to length exactly $\lfloor bn \rfloor$. Hence

$$\#\text{leaves} \;\le\; \sum_{i \le k\varepsilon^2 n} \binom{\lfloor bn \rfloor}{i} \;\le\; \left(\frac{eb}{k\varepsilon^2}\right)^{k\varepsilon^2 n} \;=\; 2^{\,k\varepsilon^2 \log_2(eb/(k\varepsilon^2))\, n}.$$

Here

$$k\varepsilon^2 \log_2\big(eb/(k\varepsilon^2)\big) = k\varepsilon^2 \cdot O\big(k \log(k/\varepsilon)\big) = \varepsilon \cdot O\big(k^2 \varepsilon \log(k/\varepsilon)\big) \le \varepsilon,$$

because ε is small enough in terms of k. So the tree has at most $2^{\varepsilon n}$ leaves. □

Claim 8 (Time)**.** *The algorithm runs in time* $2^{\varepsilon n}\mathrm{poly}_{k,\varepsilon}(n)$.

Proof. A formula has $n^{O(k)}$ clauses, so one call takes time $\mathrm{poly}_k(n)$, not counting its recursive calls. Every node is on a path from the root to a leaf, and a path has at most $bn + 1$ nodes, so the tree has at most $(bn+1) \cdot 2^{\varepsilon n}$ nodes. □

A.4 Proof of the Lemma

Proof of Lemma 1. We run Sparsify on F, with the thresholds of Section A.3, and we take for $\mathcal{L}$ the set of leaf formulas. Every clause that the algorithm adds has at most k literals on the same n variables, so every formula in $\mathcal{L}$ is a k-CNF formula on the same n variables. We now check the three properties and the time.

(a) By Claim 5, the tree has at most $2^{\varepsilon n}$ leaves, so $\mathcal{L}$ has at most $2^{\varepsilon n}$ formulas.

(b) By Claim 4, every leaf formula has at most Mn clauses.

(c) By Lemma 3, DEDUP keeps the satisfying assignments, and at each split every assignment that satisfies the formula satisfies exactly one of F_1 and F_2, while every assignment that satisfies F_1 or F_2 satisfies the formula. Going from the root down to the leaves, every assignment that satisfies F satisfies exactly one formula in $\mathcal{L}$, and every assignment that satisfies a formula in $\mathcal{L}$ satisfies F.

Time. By Claim 8, the algorithm runs in time $2^{\varepsilon n}\mathrm{poly}_{k,\varepsilon}(n)$. □

References

[1] B. Bollobás, A. D. Scott. Judicious partitions of bounded-degree graphs. *J. Graph Theory* 46(2):131–143, 2004.

[2] C. Calabro, R. Impagliazzo, R. Paturi. A duality between clause width and clause density for SAT. *Proc. 21st IEEE Conference on Computational Complexity*, pages 252–260, 2006.

[3] Claude (Anthropic). A $2^{o(n)}$ time algorithm for 3-SAT. Anthropic manuscript, October 26, 2026.

[4] H. Dell, T. Husfeldt, D. Marx, N. Taslaman, M. Wahlén. Exponential time complexity of the permanent and the Tutte polynomial. *ACM Transactions on Algorithms* 10(4), Article 21, 2014.

[5] R. Impagliazzo, R. Paturi. On the complexity of k-SAT. *J. Computer and System Sciences* 62(2):367–375, 2001.

[6] R. Impagliazzo, R. Paturi, F. Zane. Which problems have strongly exponential complexity? *J. Computer and System Sciences* 63(4):512–530, 2001.